

TUGAS 5
SISTEM OPERASI



Dosen Pembimbing:
Triawan Adi Cahyanto M.Kom

Dibuat oleh:
Darina Qothrun Nada [2410651047]

TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

SISTEM BOOKING HOTEL (Akhiran NIM Ganjil)

Deskripsi Masalah:

Anda diminta membuat simulasi sistem booking hotel dengan kondisi:

- Hotel memiliki **5 kamar** yang bisa di-booking
- Ada **10 customer** yang mencoba booking secara bersamaan (10 threads)

Setiap customer akan:

1. Mencoba booking kamar (butuh waktu random 1-3 detik)
 2. Jika berhasil, "menginap" selama 5 detik
 3. Check-out dan melepaskan kamar
 4. Jika gagal (kamar penuh), customer menunggu 2 detik lalu coba lagi
- Customer **maksimal mencoba 3 kali**, jika gagal terus maka menyerah

```
darin@DARINAGNADA:~/tugas-booking-hotel$ nano hotel_booking.c
darin@DARINAGNADA:~/tugas-booking-hotel$ gcc hotel_booking.c -o hotel -pthread
darin@DARINAGNADA:~/tugas-booking-hotel$ ./hotel

=====
SIMULASI SISTEM BOOKING HOTEL
=====
Kapasitas: 5 Kamar. Total Customer: 10
=====

[✓] Customer 1 (Coba 1) berhasil booking kamar 1!
[🏠] Sisa kamar tersedia: 4
[✓] Customer 2 (Coba 1) berhasil booking kamar 2!
[🏠] Sisa kamar tersedia: 3
[✓] Customer 3 (Coba 1) berhasil booking kamar 3!
[🏠] Sisa kamar tersedia: 2
[✓] Customer 4 (Coba 1) berhasil booking kamar 4!
[🏠] Sisa kamar tersedia: 1
[✓] Customer 5 (Coba 1) berhasil booking kamar 5!
[🏠] Sisa kamar tersedia: 0
[✗] Customer 6 (Coba 1) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 7 (Coba 1) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 8 (Coba 1) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 9 (Coba 1) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 10 (Coba 1) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 6 (Coba 2) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 7 (Coba 2) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 8 (Coba 2) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 9 (Coba 2) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 10 (Coba 2) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 6 (Coba 3) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 7 (Coba 3) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 8 (Coba 3) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 9 (Coba 3) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 10 (Coba 3) GAGAL booking: Kamar penuh! Menunggu 2 detik...
[✗] Customer 6 menyerah setelah 3 kali coba booking gagal.
[✗] Customer 7 menyerah setelah 3 kali coba booking gagal.
[✗] Customer 8 menyerah setelah 3 kali coba booking gagal.
[✗] Customer 9 menyerah setelah 3 kali coba booking gagal.
[✗] Customer 10 menyerah setelah 3 kali coba booking gagal.
[🏠] Customer 1 Check-out dari kamar 1.
[🏠] Sisa kamar tersedia: 1
[🏠] Customer 2 Check-out dari kamar 2.
[🏠] Sisa kamar tersedia: 2
[🏠] Customer 3 Check-out dari kamar 3.
[🏠] Sisa kamar tersedia: 3
[🏠] Customer 4 Check-out dari kamar 4.
[🏠] Sisa kamar tersedia: 4
[🏠] Customer 5 Check-out dari kamar 5.
[🏠] Sisa kamar tersedia: 5

=====
Selesai. Semua customer telah mencoba booking.
=====
```

Saya menggunakan dua mekanisme sinkronisasi untuk mengatasi Race Condition dan masalah concurrency :

1. Semaphore (kamar_tersedia):
 - Peran: Mengatur jumlah kamar yang tersedia (nilai awal 5). Ini adalah Counting Semaphore.
 - Mekanisme: Ketika customer mencoba booking, ia memanggil `sem_trywait()`. Jika kamar tersedia (counter > 0), customer berhasil booking. Jika kamar penuh (counter = 0), customer gagal dan harus mencoba lagi nanti. Ini mencegah lebih dari 5 thread (customer) berada dalam status "Menginap" secara bersamaan.

2. Mutex (print_lock):

- Peran: Melindungi shared resource berupa terminal output (printf) dan array status kamar (kamar_status).
- Mekanisme: Setiap kali thread (customer) ingin mencetak hasil (berhasil/gagal booking, check-out) atau mengubah array kamar_status, ia harus mengunci pthread_mutex_lock(&print_lock) terlebih dahulu. Ini memastikan output tercetak sebaris demi sebaris dan update kamar status (mengosongkan kamar) dilakukan secara atomik, sehingga informasi yang ditampilkan tidak berantakan.

Analisis:

Apa yang Terjadi Jika Tidak Menggunakan Sinkronisasi

Jika kita tidak menggunakan Semaphore dan Mutex, sistem booking akan mengalami kegagalan total:

- Tanpa Semaphore:
 - Overbooking (Race Condition): Lebih dari 5 customer akan menganggap kamar tersedia dan memasuki fase "menginap" secara bersamaan. Misalnya, dua customer membaca kamar 1 kosong, lalu keduanya sama-sama masuk. Ini melanggar batas fisik hotel (5 kamar).
- Tanpa Mutex:
 - Output Kacau (Race Condition): Output di terminal (pesan printf) akan saling tumpang tindih (garbled). Misalnya, Thread 1 mencetak separuh pesan, lalu Thread 2 mencetak separuh pesan di baris yang sama, membuat informasi tidak terbaca.
 - Status Kamar Rusak: Jika dua thread mencoba mengosongkan atau mengisi array kamar_status secara bersamaan, nilai index kamar bisa salah di-update, menyebabkan sistem menampilkan kamar yang sudah terisi sebagai kosong, atau sebaliknya.

Kesimpulannya, sinkronisasi adalah wajib untuk sistem booking hotel agar data status kamar dan proses booking berjalan sesuai kapasitas fisik dan logis.